ADVANCED JAVA — SERVLETS

Lecture 1: Introduction to Web Programming

---

1. Web Programming

Web Programming is the development of applications that communicate between a **client and a server** over a network.

Examples:
Gmail, Amazon, Instagram, Banking Applications

Basic Flow

```
Client → HTTP Request → Server
Client ← HTTP Response ← Server
```

---

2. Desktop vs Web Application

Desktop Application	Web Application
Runs on local computer	Runs through a browser
Usually requires installation	Usually no installation
Application runs locally	Server handles application logic
Updates may require reinstallation	Updates are generally server-side

---

3. Static vs Dynamic Website

Static Website

Content is **fixed** and generally does not change based on the user.

Examples: Portfolio, Company Profile

Dynamic Website

Content is **generated or modified based on user input, request, or data**.

Examples: Gmail, Amazon, Banking Applications

```
Static  → Same/FIXED Content
Dynamic → Content can change
```

---

4. Client-Server Architecture

Client

The application that **sends requests** to the server.

Examples: Chrome, Firefox, Mobile App

Server

The system that **receives requests, processes them, and sends responses**.

```
Browser
   ↓
HTTP Request
   ↓
Server
   ↓
Processing
   ↓
HTTP Response
   ↓
Browser
```

---

5. HTTP

HTTP (HyperText Transfer Protocol) is a protocol used for communication between a client and a server.

HTTP Request

Sent by the client to the server.

Example:

```
GET /students
POST /login
```

HTTP Response

Sent by the server back to the client.

Example:

```
200 OK
404 Not Found
500 Internal Server Error
```

---

6. HTTP Methods

Method	Purpose
GET	Retrieve data
POST	Send/Create data
PUT	Update/Replace data
PATCH	Partially update data
DELETE	Delete data

---

7. HTTP Status Codes

2xx — Success

```
200 → OK
201 → Created
```

3xx — Redirection

```
301 → Moved Permanently
302 → Found
```

4xx — Client Error

```
400 → Bad Request
401 → Unauthorized
403 → Forbidden
404 → Not Found
```

5xx — Server Error

```
500 → Internal Server Error
```

---

8. Web Server

A **Web Server** receives HTTP requests and provides web resources or responses.

Examples:

- Apache HTTP Server
- Nginx

---

9. Application Server

An **Application Server** provides an environment for executing server-side application logic.

It can handle:

- Business Logic
- Database Communication
- Sessions
- Security
- Application Services

---

10. Servlet Container

A **Servlet Container** is the runtime environment that manages Java Servlets.

Responsibilities:

- Creates Servlet objects
- Manages Servlet lifecycle
- Receives requests
- Sends requests to Servlets
- Provides request/response objects
- Manages sessions

Example: Apache Tomcat

---

11. Apache Tomcat

Apache Tomcat is an open-source **Servlet Container** used to run Java web applications based on Servlets and JSP.

```
Client
   ↓
Tomcat
   ↓
Servlet
   ↓
Response
```

---

12. Servlet

A **Servlet** is a Java class that runs inside a Servlet Container and processes client requests.

A Servlet can:

- Receive request data
- Process business logic
- Access a database
- Manage sessions
- Generate responses

Basic Flow

```
Browser
   ↓
HTTP Request
   ↓
Tomcat
   ↓
Servlet
   ↓
Business Logic
   ↓
Database
   ↓
Servlet
   ↓
HTTP Response
   ↓
Browser
```

---

13. Why Servlets?

Servlets provide a standard Java mechanism for developing **server-side web applications**.

They allow Java applications to:

```
Receive Request
      ↓
Process Data
      ↓
Execute Logic
      ↓
Access Database
      ↓
Generate Response
```

---

14. Servlet Advantages

- Platform Independent
- Good Performance
- Multithreaded
- Secure
- Portable
- Database Integration
- Session Management

---

15. Servlet Limitations

- HTML generation inside Java becomes difficult to maintain.
- Presentation and business logic can become tightly coupled.
- Large applications become difficult to manage using Servlets alone.

This led to technologies such as **JSP, MVC, and modern web frameworks**.

---

16. Servlet Architecture

```
            CLIENT
               ↓
         Web Browser
               ↓
         HTTP Request
               ↓
       Servlet Container
          (Tomcat)
```

```
            ↓
        Servlet
            ↓
    Business Logic
            ↓
     JDBC / Database
            ↓
        Servlet
            ↓
    HTTP Response
            ↓
     Web Browser
            ↓
        CLIENT
```

---

17. Key Terms

```
Web Application
→ Application accessed through a web browser.

Client
→ Sends requests.

Server
→ Processes requests and sends responses.

HTTP
→ Communication protocol between client and server.

Web Server
→ Handles web requests and resources.

Servlet Container
→ Manages Servlet execution and lifecycle.

Tomcat
→ Popular Java Servlet Container.

Servlet
→ Java class used to process web requests.
```

Lecture Summary

```
Web Application
      ↓
Client-Server
      ↓
HTTP
      ↓
Request / Response
      ↓
Web Server
      ↓
Servlet Container
      ↓
Tomcat
      ↓
Servlet
```

Next Lecture

Servlet Lifecycle

- Servlet Interface
- GenericServlet
- HttpServlet
- `init()`
- `service()`
- `destroy()`
- `doGet()`
- `doPost()`